Case Study- 1

Fall Semester 2025-26

NAME:- ARYAN SINGH SIKARWAR

Reg no:- 24BCE0403

Course Code: BECE102L

Course Name: Digital Systems Design

Max. Marks : 10

Class ID: VL2025260103643

Slot : F1+TF1

Case Study: Binary to Gray Code Converter

Questions:

i. What is the rule for converting binary to Gray code?

Ans-: The rule for converting a binary number to Gray code is a straightforward process involving the XOR (exclusive OR) operation. Gray code, also known as reflected binary code, is a binary numeral system where two successive values differ in only one bit. This property is particularly useful in preventing errors in digital systems.

Here's a detailed explanation of the conversion process:

The Conversion Rule

1. **Most Significant Bit (MSB):** The most significant bit (the leftmost bit) of the Gray code is the same as the most significant bit of the binary number.
2. **Subsequent Bits:** To get the remaining Gray code bits, you perform an XOR operation on the current binary bit and the binary bit to its immediate left.

Let's denote the binary bits as $B_n, B_{n-1}, \ldots, B_1, B_0$ and the corresponding Gray code bits as $G_n, G_{n-1}, \ldots, G_1, G_0$.

The conversion can be expressed with the following formulas:

- $G_n = B_n$
- $G_{n-1} = B_n \oplus B_{n-1}$
- $G_{n-2} = B_{n-1} \oplus B_{n-2}$
- ...
- $G_0 = B_1 \oplus B_0$

The $\oplus$ symbol represents the XOR operation. The truth table for XOR is:

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1

1	1	0

Export to Sheets

Step-by-Step Example

Let's convert the binary number **10110** to Gray code.

Step 1: The Most Significant Bit

The MSB of the binary number is **1**. Therefore, the MSB of the Gray code will also be **1**.

- Binary: **1**0110
- Gray: **1**

Step 2: The Second Bit

Now, we perform an XOR operation on the first and second bits of the binary number (1 and 0).

- $1 \oplus 0 = 1$
- So, the second bit of the Gray code is **1**.
- Gray: 1**1**

Step 3: The Third Bit

Next, we XOR the second and third bits of the binary number (0 and 1).

- $0 \oplus 1 = 1$
- The third bit of our Gray code is **1**.

- Gray: 11**1**

Step 4: The Fourth Bit

We continue by XORing the third and fourth bits of the binary number (1 and 1).

- $1 \oplus 1 = 0$
- The fourth bit of the Gray code is **0**.
- Gray: 111**0**

Step 5: The Fifth Bit

Finally, we XOR the fourth and fifth bits of the binary number (1 and 0).

- $1 \oplus 0 = 1$
- The last bit of the Gray code is **1**.
- Gray: 1110**1**

So, the Gray code equivalent of the binary number **10110** is **11101**.

ii. Derive the logic expressions for each output bit

Ans:- The logic expressions for converting a binary number to Gray code can be derived directly from the conversion rule. Let's denote the binary input bits as **B** and the Gray code output bits as **G**. For a 4-bit

conversion, the binary input would be B3 B2 B1 B0 and the Gray code output would be G3 G2 G1 G0 .

Here are the derived logic expressions for each output bit:

Most Significant Bit (MSB)

The rule states that the most significant bit of the Gray code is the same as the MSB of the binary input.

- **Logic Expression:** $G3 = B3$

This means the output bit G3 is simply a direct connection from the input bit B3 . No logic gate is required.

Next Significant Bit

The second Gray code bit is the result of an XOR operation between the first and second binary bits.

- **Logic Expression:** $G2 = B3 \oplus B2$

This is read as "G2 equals B3 XOR B2 ". An **XOR gate** is used to implement this logic. The output is '1'

if the inputs are different, and '0' if they are the same.

Following Bits

The pattern continues for the remaining bits. Each Gray code bit is the XOR of the corresponding binary bit and the binary bit to its immediate left.

- **Logic Expression for G1:** $G1 = B2 \oplus B1$
- **Logic Expression for G0:** $G0 = B1 \oplus B0$

Each of these expressions is implemented using a 2-input XOR gate.

General Logic Expressions

For an n-bit binary number, where 'i' represents the bit position (from 0 to n-1), the logic expressions are:

- **For the MSB (i = n-1):**

$Gi = Bi$

- **For all other bits (i < n-1):**

$Gi = Bi+1 \oplus Bi$

iv. Why is Gray code used in digital communication or error correction systems?

Ans:- **Minimizing Errors in Mechanical and Digital Systems**

The single-bit-change property of Gray code is particularly beneficial in mechanical encoders and certain digital systems.

Mechanical Encoders

Consider a mechanical encoder that uses a rotating disk to represent a position. If standard binary code were used, a transition between two adjacent positions could cause multiple bits to change simultaneously. For instance, moving from 3 (binary 011) to 4 (binary 100) requires all three bits to change.

Due to slight misalignments in the sensors or the disk itself, these bit changes might not occur at the exact same instant. This can lead to a temporary, incorrect reading. For example, during the transition from 3 to 4, the system might briefly read 7 (binary 111) if the most significant bit changes before the

others. This is known as a **glitch** or a **spurious state**.

Gray code eliminates this problem. When transitioning between any two adjacent values, only one bit changes. This ensures that even with slight timing differences, the system will only ever read one of the two adjacent correct values, preventing significant errors.

Digital Communication

In digital communication, signals are often transmitted sequentially. The single-bit-change property of Gray code can help in reducing the impact of transmission errors. If a bit is flipped during transmission, the resulting value will still be very close to the original value. This is especially useful in systems where small errors are more tolerable than large ones.

Error Detection and Correction

While Gray code itself isn't an error-correcting code in the traditional sense (like Hamming or Reed-

Solomon codes), its properties can be leveraged in error detection and some correction schemes.

The predictable single-bit change between consecutive values can be used to detect certain types of errors. If a received sequence of Gray-coded values shows a change of more than one bit between consecutive samples, it indicates a potential error in transmission or reading.

Furthermore, in some modulation schemes used in digital communication, such as **Quadrature Amplitude Modulation (QAM)**, Gray coding is used to map data bits to constellation points. This mapping ensures that if a received symbol is decoded to an adjacent constellation point due to noise, it results in only a single bit error in the demodulated data. This minimizes the bit error rate (BER) of the communication system.

v. How would you implement the reverse conversion (Gray to Binary)?

Ans:- **The Conversion Rule**

1. **Most Significant Bit (MSB):** The most significant bit (the leftmost bit) of the binary number is the same as the most significant bit of the Gray code.
2. **Subsequent Bits:** To obtain the remaining binary bits, you perform an XOR operation between the *previously calculated binary bit* and the *current Gray code bit*.

Let's denote the Gray code bits as $G_n, G_{n-1}, ..., G_1, G_0$ and the corresponding binary bits as $B_n, B_{n-1}, ..., B_1, B_0$.

The conversion can be expressed with the following formulas:

- $B_n = G_n$
- $B_{n-1} = B_n \oplus G_{n-1}$
- $B_{n-2} = B_{n-1} \oplus G_{n-2}$
- ...
- $B_0 = B_1 \oplus G_0$

The $\oplus$ symbol represents the XOR operation. Remember the XOR truth table:

Input 1	Input 2	Output
0	0	0

0	1	1
1	0	1
1	1	0

Export to Sheets

Step-by-Step Example

Let's convert the Gray code **11101** back to binary.

Step 1: The Most Significant Bit

The MSB of the Gray code is **1**. Therefore, the MSB of the binary number will also be **1**.

- Gray: **1**1101
- Binary: **1**

Step 2: The Second Bit

Now, we perform an XOR operation on the *first binary bit we just found* (1) and the second Gray code bit (1).

- $1 \oplus 1 = 0$
- So, the second bit of the binary number is **0**.
- Binary: 1**0**

Step 3: The Third Bit

Next, we XOR the *second binary bit we just calculated* (0) and the third Gray code bit (1).

- $0 \oplus 1 = 1$
- The third bit of our binary number is **1**.
- Binary: 10**1**

Step 4: The Fourth Bit

We continue by XORing the *third binary bit* (1) and the fourth Gray code bit (0).

- $1 \oplus 0 = 1$
- The fourth bit of the binary number is **1**.
- Binary: 101**1**

Step 5: The Fifth Bit

Finally, we XOR the *fourth binary bit* (1) and the fifth Gray code bit (1).

- $1 \oplus 1 = 0$
- The last bit of the binary number is **0**.
- Binary: 1011**0**

So, the binary equivalent of the Gray code **11101** is **10110**. This confirms our earlier conversion from binary to Gray. □

iii. Draw the logic circuit diagram?

Ans:- **Components Used:**

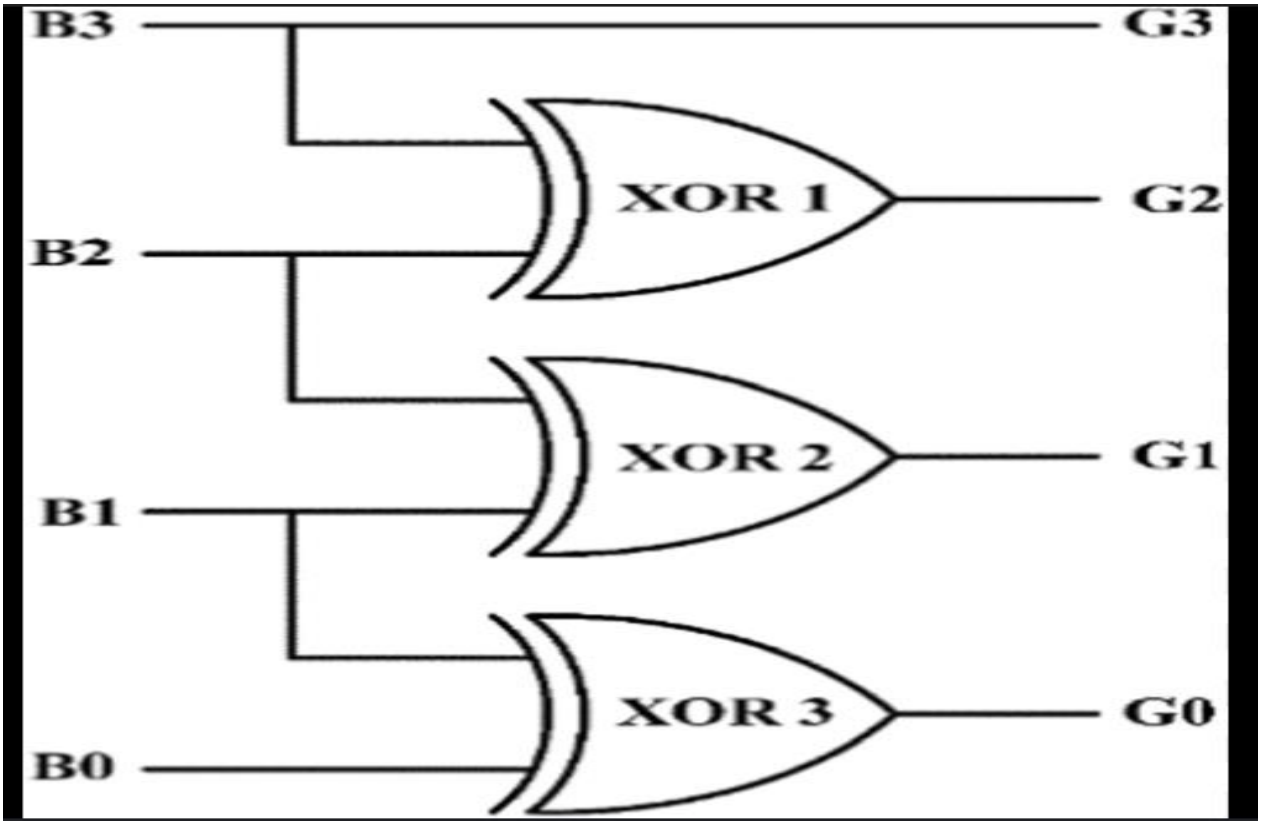
- **XOR Gates (represented by the 'X' inside the gate symbol):** These are the core logic elements. An XOR gate outputs a '1' if its inputs are different, and a '0' if its inputs are the same.
- **Input Lines:** These carry the Gray code bits (G3 ,G2 ,G1 ,G0).
- **Output Lines:** These carry the calculated binary bits (B3 ,B2 ,B1 ,B0).

Detailed Breakdown of Each Output Bit Calculation:

1. **Calculating B3 (Most Significant Bit):**
    a. **Logic Expression:** B3 =G3
    b. **Circuit Implementation:** As shown in the diagram, the input G3 is directly connected to the output B3 . No logic gate is needed for this bit, as it's simply a pass-through.
2. **Calculating B2 :**
    a. **Logic Expression:** B2 =B3 ⊕G2
    b. Circuit Implementation:

i. The previously calculated binary bit, B3 , is fed as one input to the first XOR gate.
ii. The Gray code input bit, G2 , is fed as the other input to the same XOR gate.
iii. The output of this XOR gate is B2 .

3. **Calculating B1 :**
   a. **Logic Expression:** $B1 = B2 \oplus G1$
   b. **Circuit Implementation:**
      i. The binary bit B2 (which was just calculated by the previous XOR gate) is fed as one input to the second XOR gate.
      ii. The Gray code input bit, G1 , is fed as the other input to this XOR gate.
      iii. The output of this XOR gate is B1 .
4. **Calculating B0 (Least Significant Bit):**
   a. **Logic Expression:** $B0 = B1 \oplus G0$
   b. **Circuit Implementation:**
      i. The binary bit B1 (calculated by the second XOR gate) is fed as one input to the third XOR gate.
      ii. The Gray code input bit, G0 , is fed as the other input to this XOR gate.
      iii. The output of this final XOR gate is B0 .


B3
G3
XOR 1
G2
B2
XOR 2
G1
B1
XOR 3
G0
B0